Database-System Architectures: Q & A

1 Centralized and Distributed Systems

Q1: Centralized vs. Client-Server

What are the main differences between centralized database systems and client-server architectures?

Solution

Centralized Systems: All processing (application logic and database operations) occurs on a single machine. Users access via terminals with no local processing power.
Client-Server: Processing is split between client machines (application logic, user interface) and server machines (database operations). This reduces server load and network traffic by processing queries locally when possible.

Q2: Two-Tier vs. Three-Tier Architecture

Explain the key difference between two-tier and three-tier client-server architectures. What are the advantages of three-tier?

Solution

Two-Tier: Clients directly communicate with the database server. Application logic resides on clients.
Three-Tier: Introduces an **application server** layer between clients and database. Clients communicate with the application server, which then queries the database.
Advantages of Three-Tier:

- **Centralized Business Logic:** Easier to maintain and update
- **Better Security:** Database is isolated from direct client access
- **Scalability:** Application servers can be load-balanced
- **Thin Clients:** Clients need minimal resources

Q3: Multiuser vs. Parallel Systems

Is a multiuser system necessarily a parallel system? Why or why not?

Solution

No. A multiuser system is **not** necessarily a parallel system.

Explanation: A single processor with only one core can run multiple processes to manage multiple users through **time-sharing** (context switching). The operating system rapidly switches between processes, giving each user the illusion of concurrent execution, even though only one process executes at any given instant.

Modern Reality: Most modern systems *are* parallel (multi-core processors), but the fundamental concept of multiuser support does not require parallelism—it only requires multitasking.

2 Server System Architectures

Q4: Transaction Server vs. Data Server

Compare transaction-server and data-server architectures in terms of where computation occurs and communication overhead.

Solution

Aspect	Transaction Server	Data Server
Computation	At server	At client
Data Transfer	Result sets only	Entire data items
Communication	Lower (only results)	Higher (raw data)
Use Case	Many concurrent users	Data-intensive analysis

Q5: Server Process Structure

List the key background processes in a transaction-server architecture and their roles.

Solution

- **Server Processes:** Execute database operations for client requests
- **Lock Manager:** Handles lock acquisition/release and deadlock detection
- **Database Writer:** Flushes modified buffer pages to disk
- **Log Writer:** Writes log records to stable storage
- **Checkpoint Process:** Periodically checkpoints the database
- **Process Monitor:** Detects and recovers from process failures

3 Transaction System Processes

Q6: Direct Lock Table Access

Why do modern systems allow server processes to directly access the lock table in shared memory instead of using a separate lock manager process?

Solution

Reason: To avoid the overhead of **message-passing (RPC)** for every lock request. Direct access via shared memory is much faster than inter-process communication.
Note: The lock manager process still exists but its role is limited to **deadlock detection** rather than handling every lock request.

Q7: Test-And-Set vs. Compare-And-Swap

Explain the difference between Test-And-Set (TAS) and Compare-And-Swap (CAS) atomic instructions.

Solution

Test-And-Set (TAS):

- Sets memory location M to 1 and returns the old value atomically
- Return value 0 = success (mutex acquired)
- Return value 1 = failure (mutex held by another process)

Compare-And-Swap (CAS):

- Compares M with expected value $V1$
- If equal, sets M to $V2$ and returns success
- More versatile: can use process ID instead of just 1/0
- Example: `CAS(M, 0, ProcessID)` records *who* holds the lock

Q8: Memory Barrier Instructions

What are memory barriers and why are they needed in weak consistency models?

Solution

Memory Barriers ensure cache coherency in multi-core systems:

- **sfence (Store Barrier):** Forces cached writes to be flushed to main memory
- **lfence (Load Barrier):** Ensures all pending cache invalidations are processed
- **mfence:** Performs both store and load barriers

Why Needed: In weak consistency models, processors may have stale cached data. Barriers ensure all processors see consistent, up-to-date values before critical operations (typically used automatically in locking code).

4 Data Servers and Client-Side Optimization

Q9: Client-Side Caching Strategies

Describe three client-side optimization strategies used in data-server architectures to reduce network latency.

Solution

1. **Prefetching:** Proactively fetch data items likely to be used soon, reducing future round-trips.
2. **Data Caching:** Cache data items locally (even across transactions). Must verify cache coherency before use, typically when requesting locks.
3. **Lock Caching:** Retain locks across transactions. Subsequent transactions can acquire cached locks locally. Server issues callbacks when conflicting lock requests arrive.

Q10: Adaptive Lock Granularity

Explain lock granularity escalation and de-escalation. When is each used?

Solution

Lock Escalation: Switch from many fine-grained locks (e.g., tuple locks) to fewer coarse-grained locks (e.g., page/table locks).

- **When:** To reduce lock management overhead when many locks are held

Lock De-escalation: Start with coarse-grained locks, switch to finer locks when needed.

- **When:** High concurrency conflict detected at server; finer locks improve parallelism

5 Parallel Systems

Q11: Speedup vs. Scaleup

Define speedup and scaleup. What is the difference between linear and sublinear performance?

Solution

Speedup: Running a **fixed-size task** faster by adding resources.

$$\text{Speedup} = \frac{T_{\text{small system}}}{T_{\text{large system}}}$$

Linear Speedup: Speedup $= N$ when resources increase by factor N.
Scaleup: Handling an **N-times larger task** in the same time with N-times more resources.

$$\text{Scaleup} = \frac{T_{\text{small problem}}}{T_{\text{large problem}}}$$

Linear Scaleup: Scaleup = 1 (constant execution time).
Sublinear: Performance gains less than proportional to resources added (due to overhead, skew, interference).

Q12: Amdahl's Law

State Amdahl's Law and explain its implication for parallel speedup.

Solution

Amdahl's Law: If fraction $(1 - p)$ of a task is sequential:

$$\text{Speedup} = \frac{1}{(1-p) + \frac{p}{n}}$$

Implication: Even with infinite processors, speedup is limited by the sequential portion:

$$\text{Max Speedup} = \frac{1}{1-p}$$

Example: If 10% is sequential ($p = 0.9$), maximum speedup is only $10\times$, regardless of how many processors are added.

Q13: Batch vs. Transaction Scaleup

Differentiate between batch scaleup and transaction scaleup.

Solution

Batch Scaleup:

- Large, single jobs (e.g., decision-support queries, simulations)
- Problem size = database size or computation complexity
- Example: Scanning an N-times larger relation

Transaction Scaleup:

- Many small, independent transactions
- Problem size = transaction rate
- Example: N-times more users submitting requests to N-times larger database
- Well-suited for parallel execution (transactions are independent)

6 Interconnection Networks

Q14: Network Topology Comparison

Compare Bus, Mesh, and Hypercube topologies in terms of scalability and hop count.

Solution

Topology	Scalability	Max Hops	Notes
Bus	Poor	1	Bottleneck at shared bus
Mesh	Moderate	$2\sqrt{n}$ (or $\sqrt{n}$ with wraparound)	Links grow with nodes
Hypercube	Good	$\log(n)$	Each node connects to $\log(n)$ others

Q15: Fat-Tree Topology

Describe the three-layer hierarchy of a Fat-Tree topology used in modern data centers.

Solution

Fat-Tree Layers:

1. **Top-of-Rack (ToR) Switches:** Connect $\sim$40 machines within a single rack
2. **Aggregation Switches:** Connect multiple ToR switches
3. **Core Switches:** Connect aggregation layers to form the data center fabric

Benefits: High bandwidth, redundancy, and scalability for hundreds of thousands of servers.

Q16: Infiniband vs. Ethernet

Why is Infiniband preferred over Ethernet for parallel database systems despite higher cost?

Solution

Infiniband Advantages:

- **Extremely Low Latency:** 0.5–0.7 microseconds vs. several microseconds for Ethernet
- **RDMA Support:** Remote Direct Memory Access for zero-copy data transfer
- **Critical for Synchronization:** Essential for massive scale-out systems requiring tight coordination

Trade-off: Higher cost, but latency is a fundamental bottleneck that cannot be reduced through software optimization alone.

7 Parallel Database Architectures

Q17: Shared Memory Scalability Limit

Why doesn't shared-memory architecture scale beyond 64–128 cores?

Solution

Bottleneck: The **memory interconnection network** becomes saturated. As more cores compete for memory access, contention increases exponentially, limiting effective parallelism.
Solution: Use shared-nothing or NUMA architectures for larger-scale systems.

Q18: Shared Disk Fault Tolerance

How does shared-disk architecture provide fault tolerance?

Solution

Mechanism: All processors access all disks via SAN (Storage Area Network). Each processor has private memory but shared disk access.
Fault Tolerance: If a processor fails, other processors can take over its tasks because the data resides on shared disks accessible to all nodes.
Limitation: Disk subsystem interconnection can become a bottleneck.

Q19: NUMA vs. RDMA

Explain the difference between NUMA and RDMA.

Solution

NUMA (Non-Uniform Memory Architecture):

- Shared-memory *software view* of physically distributed memory
- Each processor has local memory (fast access) and can access remote memory (slower)
- Locality of access is critical for performance

RDMA (Remote Direct Memory Access):

- Allows one node to access another node's memory *without involving the remote CPU*
- Provides shared-memory abstraction on shared-nothing hardware
- Often implemented on Infiniband for sub-microsecond latency

8 Distributed Systems

Q20: Parallel vs. Distributed Databases

What are the key differences between parallel and distributed database systems?

Solution

- **Geographic Separation:** Distributed systems span wide areas; parallel systems are within a data center
- **Network Characteristics:** Distributed systems face higher latency, lower bandwidth, higher failure rates
- **Administration:** Distributed systems often have separate administrators per site (autonomy)
- **Disaster Recovery:** Distributed systems must handle entire data center failures
- **Node Heterogeneity:** Distributed nodes vary more in capacity and function

Q21: Local vs. Global Transactions

Define local and global transactions in distributed databases.

Solution

Local Transaction: Accesses data only from the site where it was initiated. No network communication required.
Global Transaction: Accesses data from different sites or multiple sites. Requires distributed coordination and network communication.

Q22: Two-Phase Commit Protocol

Explain the two phases of the 2PC protocol and why it's necessary.

Solution

Purpose: Ensures atomicity for transactions updating data at multiple sites.
Phase 1 (Prepare):

- Each site executes the transaction until just before commit
- Sites vote "ready" or "abort" to the coordinator

Phase 2 (Commit/Abort):

- Coordinator makes final decision based on votes
- All sites must follow the coordinator's decision
- Sites must wait even if there are failures

Limitation: Blocking protocol—sites must wait for coordinator decision.

Q23: Network Partitions

What is a network partition and how does it affect distributed databases?

Solution

Network Partition: A failure where two alive sites cannot communicate with each other (e.g., due to link failure).
Impact on Availability:

- Users may not be able to access required data
- System must choose between consistency and availability
- Creates fundamental trade-offs in distributed system design

Mitigation: Data replication across partitions, eventual consistency models.

9 Cloud-Based Services

Q24: IaaS vs. PaaS vs. SaaS

Compare the three cloud service models in terms of what the provider manages.

Solution

Model	Provider Manages	Client Manages
IaaS	Hardware, VMs/Containers	OS, Database, Applications
PaaS	Hardware, OS, Database	Applications only
SaaS	Everything	Nothing (just uses interface)

Q25: Containers vs. Virtual Machines

What are the key advantages of containers over virtual machines?

Solution

Container Advantages:

- **Lower Overhead:** Share OS kernel instead of running full OS per instance
- **Higher Density:** More containers per physical machine
- **Faster Startup:** Quick deployment for elasticity
- **Isolation:** Each has own IP address and libraries

VM Advantage: Stronger isolation (full OS separation) for multi-tenant security.

Q26: Kubernetes Features

What key features does Kubernetes provide for microservices deployment?

Solution

- **Declarative Deployment:** Specify container needs; Kubernetes handles deployment
- **Pods:** Multiple containers sharing storage and network
- **Elasticity Management:** Automatic scaling based on load
- **Load Balancing:** Distributes requests across container replicas
- **Service Discovery:** Single IP address for accessing replicated services

Q27: Cloud Security and Legal Concerns

What are the main security and legal risks of cloud computing?

Solution

Security Risks:

- Client data held by third party
- No direct control over vendor security
- Potential data breaches affecting multiple clients

Legal Risks:

- **Jurisdiction Issues:** Data may be stored in foreign countries
- **Privacy Laws:** Different countries have different regulations (EU vs. US)
- **Government Access:** Vendors may be required to release data to authorities
- **Compliance:** Difficult to ensure regulatory compliance (e.g., GDPR, HIPAA)

Mitigation: Choose vendors offering geographic control, encryption, compliance certifications.

10 Additional Practice Exercises

Q28: Memory Fence in Atomic Instructions

Atomic instructions such as compare-and-swap and test-and-set execute a memory fence as part of the instruction. Explain the motivation for executing the memory fence from the viewpoint of data in shared memory protected by a mutex. Also explain what a process should do before releasing a mutex.

Solution

Motivation: The memory fence ensures that the process acquiring the mutex will see **all updates** that happened before the instruction, as long as processes execute a fence before releasing the mutex.
Guarantee: Even if data was updated on a different core, the process that acquires the mutex is guaranteed to see the latest value of the data.
Before Releasing Mutex: A process should execute a memory fence (sfence) to ensure all its updates are visible to other processors before releasing the mutex.

Q29: Shared Memory vs. IPC for Lock Management

Instead of storing shared structures in shared memory, an alternative would be to store them in the local memory of a special process and access shared data by interprocess communication. What would be the drawbacks and benefits of such an architecture?

Solution

Drawbacks:

- **Two Messages Required:** Lock acquisition requires two IPC messages (request + grant confirmation)
- **Higher Cost:** IPC is much more expensive than memory access
- **Bottleneck:** The process storing shared structures could become a bottleneck

Benefit:

- **Better Protection:** Lock table is protected from erroneous updates since only one process can access it

Q30: Latch vs. Lock

Explain the distinction between a latch and a lock as used for transactional concurrency control.

Solution

Latches:

- **Short-duration locks** managing access to internal system data structures
- Not covered by two-phase locking protocol
- Held for very brief periods (microseconds)

Locks:

- Taken by transactions on **database data items**
- Often held for a substantial fraction of transaction duration
- Governed by two-phase locking protocol

Q31: Partial Parallelism Speedup

Suppose a transaction is written in C with embedded SQL, and about 80% of the time is spent in SQL code, with the remaining 20% in C code. How much speedup can one hope to attain if parallelism is used only for the SQL code?

Solution

Using **Amdahl's Law**: Since 20% cannot be parallelized, the best speedup is limited to:

$$\text{Max Speedup} = \frac{1}{1-p} = \frac{1}{0.2} = 5$$

Explanation: With $p = 0.8$ (parallelizable fraction) and $n \to \infty$:

$$\text{Speedup} = \frac{1}{(1-p) + \frac{p}{n}} = \frac{1}{0.2 + 0} = 5$$

No matter how many processors are used for the SQL code, the sequential C code (20%) fundamentally limits speedup to 5×.

Q32: Producer-Consumer Memory Ordering

Consider a pair of processes where process A updates a data structure and sets a flag, while process B monitors the flag and processes the data structure only after finding the flag set. Explain the problems that could arise with write reordering, and how sfence and lfence can solve this.

Solution

Problem: Out-of-order writes to main memory can result in process B seeing **some but not all** updates to the data structure, even after the flag has been set.
Solution:

- **Producer (Process A):** Issue `sfence` after updates but *before* setting the flag. Optionally issue another `sfence` after setting the flag to push the update with minimum delay.
- **Consumer (Process B):** Issue `lfence` *after* finding the flag set, but *before* accessing the data structure.

Result: Process B is guaranteed to see all updates made by Process A.

Q33: Variable Memory Access Time

In a shared-memory architecture, why might the time to access a memory location vary depending on the memory location being accessed?

Solution

NUMA Architecture: A processor can access its **own local memory** faster than it can access shared memory associated with another processor.
Reason: Time is required to transfer data between processors across the interconnection network. Local memory access avoids this overhead.

Q34: OS Optimizations for NUMA

Most operating systems for parallel machines (i) allocate memory in a local memory area when a process requests memory, and (ii) avoid moving a process from one core to another. Why are these optimizations important with NUMA?

Solution

1. **Local Memory Allocation:** If process data resides in local memory, execution is faster than if memory is non-local (remote access requires inter-processor communication).

2. **Avoiding Process Migration:** If a process moves from one core to another, it loses the benefits of local memory allocation and is forced to carry out many memory accesses from other cores, degrading performance.

Goal: Maximize locality to minimize remote memory access overhead.

Q35: Superlinear Speedup

Some database operations such as joins can see a significant difference in speed when data fits in memory compared to when it doesn't. Show how this fact can explain superlinear speedup, where an application sees speedup greater than the resources allocated.

Solution

Example: Suppose we double the main memory, and as a result, one of the relations now fits entirely in memory.
With More Memory:

- Use nested-loop join with inner relation entirely in memory
- Incur disk accesses for reading input relations only **once**

With Original Memory:

- Best join strategy may require reading a relation from disk **multiple times**

Result: Speedup $> 2\times$ despite only doubling resources, because the algorithm fundamentally changes from disk-bound to memory-bound.

Q36: Homogeneous vs. Federated Systems

What is the key distinction between homogeneous and federated distributed database systems?

Solution

Homogeneous Systems:

- Share a **common global schema**
- Run the **same database software**
- Actively cooperate on query processing
- High degree of centralized control

Federated Systems:

- May have **distinct schemas and software**
- Cooperate in only a **limited manner**
- Each site retains significant autonomy
- Lower degree of cooperation

Q37: IaaS vs. Higher-Level Cloud Services

Why might a client choose to subscribe only to the basic infrastructure-as-a-service model rather than to services offered by other cloud service models?

Solution

Reasons for Choosing IaaS Only:

- **Application Control:** Client wishes to control its own applications and not use SaaS
- **Database Choice:** Client wants to choose and manage its own database system rather than use PaaS
- **Flexibility:** Maximum control over the entire software stack
- **Custom Requirements:** Specific software or configuration needs not met by higher-level services

Q38: Virtual Machines for Database Systems

Why do cloud-computing services support traditional database systems best by using a virtual machine, instead of running directly on the service provider's actual machine?

Solution

Availability Benefits:

- If a physical machine fails, VMs running on it can be **restarted quickly** on other physical machines
- Improves overall system availability
- Minimizes downtime during hardware failures or upgrades

Assumption: Data remains accessible through:

- Multiple copies of data (replication)
- Highly available external storage systems

Additional Benefits: Isolation, resource management, and easier migration between physical hosts.

Mastering these concepts is essential for designing modern, scalable database systems.